Test report: DM maintenance scripts

Item	Value
Scripts	`main.sql`, `scripta_partitioned.sql`, `scriptb_nonpartitioned.sql`
Server	Ubuntu test server (PostgreSQL 16.15 (Ubuntu))
Database	`testdb`
Run	2026-10-22 11:56:21 to 2026-10-22 12:03:40 (server time)

16 of 16 scenarios passed

Before every scenario `tests/reset_data.sql` recreates the dummy data (schemas `dm` and `maint`), and any extra data the scenario needs is inserted. `main.sql` is then run with the answers shown, and the real result is checked with SQL queries against the tables. Rerun everything with `PGHOST=<server> PGUSER=postgres ./tests/run_tests.sh`.

Summary

ID	Scenario	Expected	Result
S01	Normal table: keep rows	2,000 rows left, none with another statusid; backup table holds 2,000; log SUCCESS.	PASS
S02	Partitioned table: remove rows is_removed = 'Y'	Only the 3,000 'Y' rows deleted; the 2,000 'N' rows stay; log SUCCESS.	PASS
S03	Wrong database is refused	Message 'STOP: not the TEST database', exit code 3, testdb unchanged, nothing logged.	PASS
S04	Table not found is refused	Message 'STOP: table not found', exit code 3, nothing changed or logged.	PASS
S05	No YES = cancelled	Message 'Cancelled - nothing was changed', no backup table, nothing logged.	PASS
S06	Keep rule matching 0 rows is refused	'Copy check failed: copied 0', table still 5,000 rows, no backup left, log FAILED.	PASS
S07	Invalid keep rule is rolled back	Error 'column does not exist', table unchanged, no backup left, log FAILED.	PASS
S08	Foreign key: TRUNCATE fails, all rolled back	TRUNCATE refused, whole run undone: 5,000 rows, no backup left, log FAILED.	PASS
S09	Second run on the same day fails safely	Second run stops: backup table already exists; table and backup keep 2,000 rows; log FAILED.	PASS
S10	Identity, generated column, index, trigger preserved	200 rows, original ids kept, generated values correct, 2 valid indexes, 1 trigger, trigger fired on reload (1,000 + 200 audit rows), next id 1001; log SUCCESS.	PASS
S11	Table name with a space	400 rows kept, backup dm."maintenance_Order Items_<date>" holds 400; log SUCCESS.	PASS
S12	Condition matching no row: nothing removed	'removed 0 rows', all rows unchanged (3,000 + 2,000); log SUCCESS.	PASS
S13	Mixed partitions: only matching rows removed	500 'Y' rows deleted from both partitions, 500 'N' rows stay, both partitions still have rows; log SUCCESS.	PASS
S14	No count(*) on the main table	Counter +3: 1 = the copy step, 2 = TRUNCATE rebuilding the 2 indexes on the empty table. No count(*) on the table.	PASS
S15	Condition removing every row is refused	'The condition removes every row' - the delete is undone: 3,000 + 2,000 rows stay; log FAILED.	PASS
S16	Invalid remove condition is rolled back	Error 'column does not exist', all rows unchanged; log FAILED.	PASS

Scenario details

S01. Normal table: keep rows: PASS

Item	Detail
Dummy data	reset_data.sql: dm.filerequests 5,000 rows, statusid 1-5 (2,000 with statusid 1 or 2).
Answers typed	`dm / filerequests / statusid IN (1, 2) / YES`
Expected	2,000 rows left, none with another statusid; backup table holds 2,000; log SUCCESS.
Checked	rows left | rows with wrong statusid | rows in backup | log status
Expected values	`2000|0|2000|SUCCESS`
Actual values	`2000|0|2000|SUCCESS`
Exit code	0

```
NOTICE:  dm.filerequests: about 5000 rows, 2 indexes, 0 triggers
NOTICE:  dm.filerequests: kept 2000, removed about 3000, indexes and triggers OK
dm.filerequests | statusid IN (1, 2) | SUCCESS | 00:00:00.843035 | kept 2000, removed about 3000, backup in
    dm.maintenance_filerequests_20261022
```

S02. Partitioned table: remove rows is_removed = 'Y': PASS

Item	Detail
Dummy data	reset_data.sql: dm.files LIST partitioned by is_removed: files_removed ('Y') 3,000, files_active ('N') 2,000, files_other (DEFAULT) 0.
Answers typed	`dm / files / is_removed = 'Y' / YES`
Expected	Only the 3,000 'Y' rows deleted; the 2,000 'N' rows stay; log SUCCESS.
Checked	files_removed | files_active | 'Y' rows left | log status
Expected values	`0|2000|0|SUCCESS`
Actual values	`0|2000|0|SUCCESS`
Exit code	0

```
NOTICE:  dm.files: about 5000 rows, about 3000 to remove
NOTICE:  dm.files: removed 3000 rows matching (is_removed = 'Y')
dm.files | is_removed = 'Y' | SUCCESS | 00:00:00.842986 | removed 3000 rows matching (is_removed = 'Y')
```

S03. Wrong database is refused: PASS

Item	Detail
Dummy data	reset_data.sql in testdb. The script is run while connected to database postgres.
Answers typed	`(stops before any question)`
Expected	Message 'STOP: not the TEST database', exit code 3, testdb unchanged, nothing logged.
Checked	message shown | rows in testdb | log
Expected values	`yes|5000|no log`
Actual values	`yes|5000|no log`
Exit code	3

```
*** STOP: not the TEST database ***
ERROR:  division by zero
```

S04. Table not found is refused: PASS

Item	Detail
Dummy data	reset_data.sql.
Answers typed	`dm / no_such_table`
Expected	Message 'STOP: table not found', exit code 3, nothing changed or logged.
Checked	message shown | rows | log
Expected values	`yes|5000|no log`
Actual values	`yes|5000|no log`
Exit code	3

```
*** STOP: table not found ***
ERROR:  division by zero
```

S05. No YES = cancelled: PASS

Item	Detail
Dummy data	reset_data.sql.
Answers typed	`dm / filerequests / statusid IN (1, 2) / no`
Expected	Message 'Cancelled - nothing was changed', no backup table, nothing logged.
Checked	message shown | rows | backup | log
Expected values	`yes|5000|no backup|no log`
Actual values	`yes|5000|no backup|no log`
Exit code	0

```
Cancelled - nothing was changed.
```

S06. Keep rule matching 0 rows is refused: PASS

Item	Detail
Dummy data	reset_data.sql (no row has statusid 99).
Answers typed	`dm / filerequests / statusid IN (99) / YES`
Expected	'Copy check failed: copied 0', table still 5,000 rows, no backup left, log FAILED.
Checked	message shown | rows | backup | log status
Expected values	`yes|5000|no backup|FAILED`
Actual values	`yes|5000|no backup|FAILED`
Exit code	0

```
NOTICE:  dm.filerequests: about 5000 rows, 2 indexes, 0 triggers
ERROR:  Copy check failed: copied 0, found 0 in dm.maintenance_filerequests_20261022
dm.filerequests | statusid IN (99) | FAILED | 00:00:00.831421 | Copy check failed: copied 0, found 0 in
    dm.maintenance_filerequests_20261022
```

S07. Invalid keep rule is rolled back: PASS

Item	Detail
Dummy data	reset_data.sql.
Answers typed	`dm / filerequests / no_such_column = 1 / YES`
Expected	Error 'column does not exist', table unchanged, no backup left, log FAILED.
Checked	message shown | rows | backup | log status
Expected values	`yes|5000|no backup|FAILED`
Actual values	`yes|5000|no backup|FAILED`
Exit code	0

```
NOTICE:  dm.filerequests: about 5000 rows, 2 indexes, 0 triggers
ERROR:  column "no_such_column" does not exist
dm.filerequests | no_such_column = 1 | FAILED | 00:00:00.855326 | column "no_such_column" does not exist
```

S08. Foreign key: TRUNCATE fails, all rolled back: PASS

Item	Detail
Dummy data	reset_data.sql + table dm.child with a foreign key to dm.filerequests.
Answers typed	`dm / filerequests / statusid IN (1, 2) / YES`
Expected	TRUNCATE refused, whole run undone: 5,000 rows, no backup left, log FAILED.
Checked	message shown | rows | backup | log status
Expected values	`yes|5000|no backup|FAILED`
Actual values	`yes|5000|no backup|FAILED`
Exit code	0

```
NOTICE:  dm.filerequests: about 5000 rows, 2 indexes, 0 triggers
ERROR:  cannot truncate a table referenced in a foreign key constraint
dm.filerequests | statusid IN (1, 2) | FAILED | 00:00:00.845973 | cannot truncate a table referenced in a foreign key
    constraint
```

S09. Second run on the same day fails safely: PASS

Item	Detail
Dummy data	reset_data.sql, then a first successful run (2,000 rows kept).
Answers typed	`dm / filerequests / statusid IN (1, 2) / YES (twice)`
Expected	Second run stops: backup table already exists; table and backup keep 2,000 rows; log FAILED.
Checked	message shown | rows | rows in backup | log status
Expected values	`yes|2000|2000|FAILED`
Actual values	`yes|2000|2000|FAILED`
Exit code	0

```
NOTICE:  dm.filerequests: about 2000 rows, 2 indexes, 0 triggers
ERROR:  relation "maintenance_filerequests_20261022" already exists
dm.filerequests | statusid IN (1, 2) | FAILED | 00:00:00.94026 | relation "maintenance_filerequests_20261022" already
    exists
```

S10. Identity, generated column, index, trigger preserved: PASS

Item	Detail
Dummy data	reset_data.sql + setup_orders.sql: dm.orders 1,000 rows with an identity id, generated column amount_x2, 2 indexes, a row trigger writing to dm.order_audit.
Answers typed	`dm / orders / statusid = 1 / YES`
Expected	200 rows, original ids kept, generated values correct, 2 valid indexes, 1 trigger, trigger fired on reload (1,000 + 200 audit rows), next id 1001; log SUCCESS.
Checked	rows | ids kept | generated OK | valid indexes | triggers | audit rows | next id | log status
Expected values	`200|t|t|2|1|1200|1001|SUCCESS`
Actual values	`200|t|t|2|1|1200|1001|SUCCESS`
Exit code	0

```
NOTICE:  dm.orders: about 1000 rows, 2 indexes, 1 triggers
NOTICE:  dm.orders: kept 200, removed about 800, indexes and triggers OK
dm.orders | statusid = 1 | SUCCESS | 00:00:00.912033 | kept 200, removed about 800, backup in
    dm.maintenance_orders_20261022
```

S11. Table name with a space: PASS

Item	Detail
Dummy data	reset_data.sql + dm."Order Items" 1,000 rows, statusid 1-5.
Answers typed	`dm / Order Items / statusid IN (1, 2) / YES`
Expected	400 rows kept, backup dm."maintenance_Order Items_<date>" holds 400; log SUCCESS.
Checked	rows | rows in backup | log status
Expected values	`400|400|SUCCESS`
Actual values	`400|400|SUCCESS`
Exit code	0

```
NOTICE:  dm."Order Items": about 1000 rows, 1 indexes, 0 triggers
NOTICE:  dm."Order Items": kept 400, removed about 600, indexes and triggers OK
dm."Order Items" | statusid IN (1, 2) | SUCCESS | 00:00:00.839004 | kept 400, removed about 600, backup in
    dm."maintenance_Order Items_20261022"
```

S12. Condition matching no row: nothing removed: PASS

Item	Detail
Dummy data	reset_data.sql (no row has is_removed = 'X').
Answers typed	`dm / files / is_removed = 'X' / YES`
Expected	'removed 0 rows', all rows unchanged (3,000 + 2,000); log SUCCESS.
Checked	message shown | files_removed | files_active | log status
Expected values	`yes|3000|2000|SUCCESS`
Actual values	`yes|3000|2000|SUCCESS`
Exit code	0

```
NOTICE:  dm.files: about 5000 rows, about 1 to remove
NOTICE:  dm.files: removed 0 rows matching (is_removed = 'X')
dm.files | is_removed = 'X' | SUCCESS | 00:00:00.868457 | removed 0 rows matching (is_removed = 'X')
```

S13. Mixed partitions: only matching rows removed: PASS

Item	Detail
Dummy data	reset_data.sql + dm.events 1,000 rows, PARTITION BY RANGE (created_on), 2 yearly partitions; both partitions hold 'Y' and 'N' rows (500 'Y').
Answers typed	`dm / events / is_removed = 'Y' / YES`
Expected	500 'Y' rows deleted from both partitions, 500 'N' rows stay, both partitions still have rows; log SUCCESS.
Checked	rows left | 'Y' rows left | both partitions still have rows | log status
Expected values	`500|0|t|SUCCESS`
Actual values	`500|0|t|SUCCESS`
Exit code	0

```
NOTICE:  dm.events: about 1000 rows, about 500 to remove
NOTICE:  dm.events: removed 500 rows matching (is_removed = 'Y')
dm.events | is_removed = 'Y' | SUCCESS | 00:00:00.889682 | removed 500 rows matching (is_removed = 'Y')
```

S14. No count(*) on the main table: PASS

Item	Detail
Dummy data	reset_data.sql. PostgreSQL's full-read counter (pg_stat_user_tables.seq_scan) is read before and after.
Answers typed	`dm / filerequests / statusid IN (1, 2) / YES`
Expected	Counter +3: 1 = the copy step, 2 = TRUNCATE rebuilding the 2 indexes on the empty table. No count(*) on the table.
Checked	full reads | log status
Expected values	`3|SUCCESS`
Actual values	`3|SUCCESS`
Exit code	0

```
NOTICE:  dm.filerequests: about 5000 rows, 2 indexes, 0 triggers
NOTICE:  dm.filerequests: kept 2000, removed about 3000, indexes and triggers OK
dm.filerequests | statusid IN (1, 2) | SUCCESS | 00:00:00.866434 | kept 2000, removed about 3000, backup in
    dm.maintenance_filerequests_20261022
```

S15. Condition removing every row is refused: PASS

Item	Detail
Dummy data	reset_data.sql (every row of dm.files is 'Y' or 'N').
Answers typed	`dm / files / is_removed IN ('Y', 'N') / YES`
Expected	'The condition removes every row' - the delete is undone: 3,000 + 2,000 rows stay; log FAILED.
Checked	message shown | files_removed | files_active | log status
Expected values	`yes|3000|2000|FAILED`
Actual values	`yes|3000|2000|FAILED`
Exit code	0

```
NOTICE:  dm.files: about 5000 rows, about 5000 to remove
ERROR:  The condition removes every row of dm.files - all undone, nothing changed
dm.files | is_removed IN ('Y', 'N') | FAILED | 00:00:00.801008 | The condition removes every row of dm.files - all
    undone, nothing changed
```

S16. Invalid remove condition is rolled back: PASS

Item	Detail
Dummy data	reset_data.sql.
Answers typed	`dm / files / no_such_column = 'Y' / YES`
Expected	Error 'column does not exist', all rows unchanged; log FAILED.
Checked	message shown | files_removed | files_active | log status
Expected values	`yes|3000|2000|FAILED`
Actual values	`yes|3000|2000|FAILED`
Exit code	0

```
ERROR:  column "no_such_column" does not exist
dm.files | no_such_column = 'Y' | FAILED | 00:00:00.845012 | column "no_such_column" does not exist
```

Observations

- **Exit code on a failed run:** when the maintenance itself fails (S06-S09, S15, S16), psql still ends with exit code 0, because `main.sql` catches the error to write `FAILED` to `maint.maintenance_log`. Check the `status` in the final output or the log, not only the exit code. The safety stops before any change (S03, S04) end with exit code 3.
- **DELETE and disk space:** on partitioned tables the matching rows are deleted. The space is reused for new rows, but the files only shrink after `VACUUM` (autovacuum runs it later on its own).
- **Network:** each connection to the server took about 2 seconds. The test runner uses keepalives, a 90-second limit per call and up to 3 attempts per scenario. In this run S15 lost its connection once and passed on the second attempt, from a fresh reset.

Not covered

- **Table locked by another session:** the scripts have no lock timeout, so they wait until the other session finishes. Stop applications before a real run.
- **Production-size data:** these runs use 5,000 rows. Timing for the outage window must be measured on a production-size copy.
- **DBeaver:** these scripts use psql commands (`\prompt`, `\if`) and are tested with psql only.